



1-2. AI & 기계학습 방법

목차

1. AI&기계학습 방법론1 선형회귀 (Linear Regression)
 1. 선형 회귀(Linear Regression)
 - 1-1. 선형 회귀(Linear Regression)
 2. 단순 선형회귀 (Simple Linear Regression)
 - 2-1. 단일 설명 변수를 이용한 단순 선형 회귀
 - 2-2. 최소 제곱법(least squares)에 의한 계수 추정
 - 2-3. 판매량과 TV 광고비와의 관계의 선형성 증명(예시)
 3. 다중선형회귀(Multiple Linear Regression)
 - 3-1. 복수 설명 변수를 이용한 다중 선형 회귀
 - 3-2. 다중선형회귀의 추정과 예측
 - 3-3. 다중선형회귀 최소제곱 해의 유도
 - 3-4. 다중선형회귀 결과 : 시각화
 - 3-5. 다중선형회귀 결과 : 광고 데이터
2. AI & 기계학습 방법론2 로지스틱 회귀(Logistic Regression)
 1. 분류
 - 1-1. 분류(Classification)란
 - 1-2. 예시: 신용카드 연체(Default)
 - 1-3. 분류 문제에 선형 회귀를 써도 될까?
 - 1-4. 다중범주에서 선형회귀는 불가능
 2. 로지스틱 회귀
 - 2-1. 로지스틱(Logistic) 회귀의 모형식
 - 2-2. MLE 활용 모수 추정
 - 2-3. 로지스틱 회귀 결과 : 신용카드 연체 데이터
3. AI&기계학습 방법론 3 신경망모델 (Neural Network)
 1. 선형 회귀
 - 1-1. 1D 선형회귀
 - 1-2. 1D 선형회귀 학습
 2. Shallow 네트워크
 - 2-1. Shallow 네트워크 vs 1D 선형회귀
 - 2-2. Shallow 네트워크 : 활성화 함수
 - 2-3. Shallow 네트워크 : 모수(parameter)
 - 2-4. Shallow 네트워크 : piecewise linear 함수
 - 2-5. Shallow 네트워크 : Hidden Units
 - 2-6. Shallow 네트워크 : 각 단계별 계산
 - 2-7. 네트워크 도식화
 3. Shallow 네트워크의 표현력
 - 3-1. 더 많은 Hidden Unit 가능
 - 3-2. Hidden Unit을 많이 두면
 - 3-3. Universal approximation theorem(보편 근사정리)
 4. 다중 출력/입력
 - 4-1. 2개 출력
 - 4-2. 2개 입력
 - 4-3. 2개 입력: 단계별 계산
 - 4-4. 2개 입력 : From Input to Output
 - 4-5. 임의의 개수의 입력, Hidden Unit, 출력
 5. Deep 네트워크
 - 5-1. 2개의 Shallow 네트워크를 하나로 합성에서 시작
 - 5-2. 6개의 Hidden Units의 Shallow NN과 비교
 - 5-3. 2개의 네트워크를 하나로 합성
 - 5-3. 2개의 네트워크를 하나로 합성
 - 5-4. 새로운 변수 활용
 - 5-5. 2층 네트워크
 - 5-6. 2층 네트워크를 하나의 식으로 표현
 - 5-7. 2개 output의 shallow 네트워크

1. AI&기계학습 방법론1 선형회귀 (Linear Regression)

학습 목표

- 선형 회귀의 핵심 개념 이해 : 입력 - 출력의 선형 관계, 잔차, 최소제곱의 직관
- 회귀계수 해석과 불확실성 파악 : 표준오차, 신뢰구간, 가설검정 결과 읽기
- 모델 적합도 평가 : 잔차 표준오차와 결정계수 등 지표로 성능 비교, 설명
- 다중선형회귀로 확장하며 해석상 주의점 이해: 변수 상관, 다중공선성, 상관과 인과의 구분

1. 선형 회귀(Linear Regression)

1-1. 선형 회귀(Linear Regression)

- 지도 학습의 가장 기초가 되는 접근중 하나, Y 가 $X_1, X_2, \dots, X_p$ 에 선형적으로 의존한다고 가정한다.
- 개념적으로도 실무적으로도 매우 유
ex) 광고비와 매출 사이의 관계

2. 단순 선형회귀 (Simple Linear Regression)

2-1. 단일 설명 변수를 이용한 단순 선형 회귀

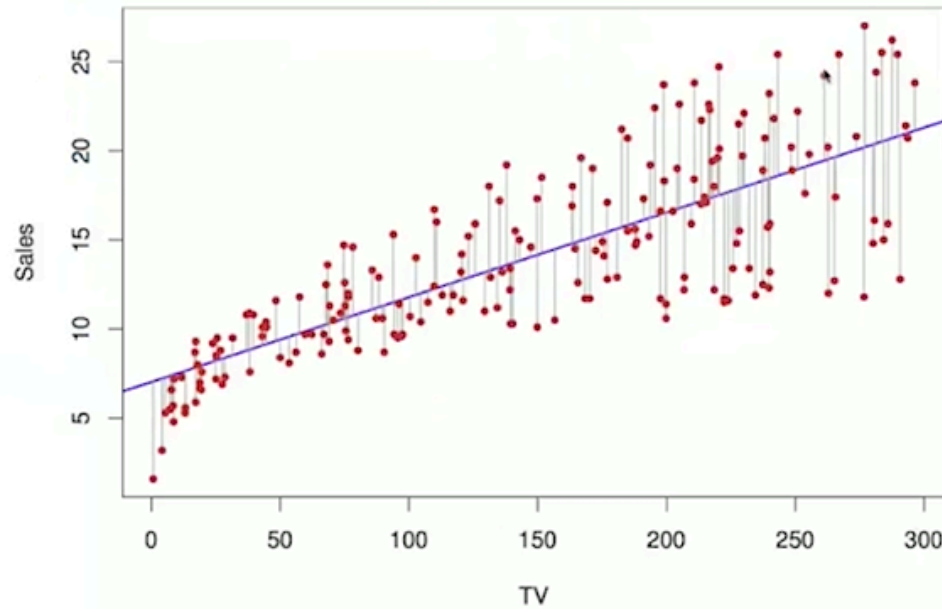
- 모형 : $Y = B_0 + B_1X + e$
 - 여기서 B_0, B_1 은 절편과 기울기 모수, e 는 관측 오차
- 추정치 $\hat{B}_0, \hat{B}_1$ 이 주어지면 $X = x$ 에서의 예측,
 - $\hat{Y} = \hat{B}_0 + \hat{B}_1 x$
- $\hat{}$ 표기는 추정값을 의미

2-2. 최소 제곱법(least squares)에 의한 계수 추정

- i 번째 관측에서의 예측
 - $\hat{y}_i = \hat{B}_0 + \hat{B}_1 * x_i$
- 잔차(residual)의 정의 : $e_i = y_i - \hat{y}_i$
- RSS(잔차제곱합) 정의
 - $RSS = e_1^2 + e_2^2 + \dots + e_n^2$
 - $= \sum (e_i^2)$
 - $= \sum (y_i - \hat{y}_i)^2$
 - $= \sum (y_i - (\hat{B}_0 + \hat{B}_1 * x_i))^2$
- 최소제곱법(least squares)는 RSS를 최소화 하는 $\hat{B}_0, \hat{B}_1$ 을 고르는 것

2-3. 판매량과 TV 광고비와의 관계의 선형성 증명(예시)

- Sales를 TV 광고비로 회귀시킨 **최소제곱 적합(RSS 최소)**



$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

- 최소제곱법(RSS 최소): 데이터(빨간 점)와 회귀직선(파란 선) 사이의 오차 제곱합을 최소화하는 직선을 찾음.
- 그림에서 볼 수 있듯, 광고비가 많을수록 Sales도 증가하는 경향이 있음.

통계적 유의성 (p-value)

- 두 계수 모두 p-value < 0.0001 → 귀무가설("관계가 없다")을 기각
→ 즉, TV 광고비와 매출 사이에 유의한 관계가 존재

모형의 설명력

- $R^2 = 0.612$ → 매출 변동의 약 61.2%를 TV 광고비로 설명 가능
- F-statistic = 312.1 → 모형이 통계적으로 유의미함

	Coefficient	Std. Error	t-statistic	p-value
Intercept	7.0325	0.4578	15.36	< 0.0001
TV	0.0475	0.0027	17.67	< 0.0001

Quantity	Value
Residual Standard Error	3.26
R^2	0.612
F-statistic	312.1

3. 다중선형회귀(Multiple Linear Regression)

3-1. 복수 설명 변수를 이용한 다중 선형 회귀

- 피쳐 (설명변수) 여러 개 존재 : $X_1, X_2, \dots, X_p$
- 모형 가정 : $Y = B_0 + B_1X_1 + B_2X_2 + \dots + B_pX_p + e$
 - 여기서 B_0, B_1 은 절편과 기울기 모수, e 는 관측 오차
- 해석 : 다른 변수를 고정한 채 X_j 가 1 단위 증가할 때, Y 가 평균적으로 B_j 만큼 변화

3-2. 다중선형회귀의 추정과 예측

- 추정치 $B_{hat0}, B_{hat1}, \dots, B_{hatp}$ 로 예측
 - i 번째 관측 ($x_{i1}, x_{i2}, \dots, x_{ip}, y_i$) 에 대한 예측

- $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \dots + \hat{\beta}_p x_{ip}$
- 최소제곱은 다음 잔차제곱합을 최소화
 - $RSS = \sum (e_i)^2 = \sum (y_i - \hat{y}_i)^2$
 - $= \sum (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \dots + \hat{\beta}_p x_{ip}))^2$

$$RSS = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \dots - \hat{\beta}_p x_{ip})^2$$

- 이를 최소화 하려는 $\hat{\beta}_0, \dots, \hat{\beta}_p$ 가 다중 최소제곱 추정치

3-3. 다중선형회귀 최소제곱 해의 유도

- 행렬 표기

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1p} \\ 1 & x_{21} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{np} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

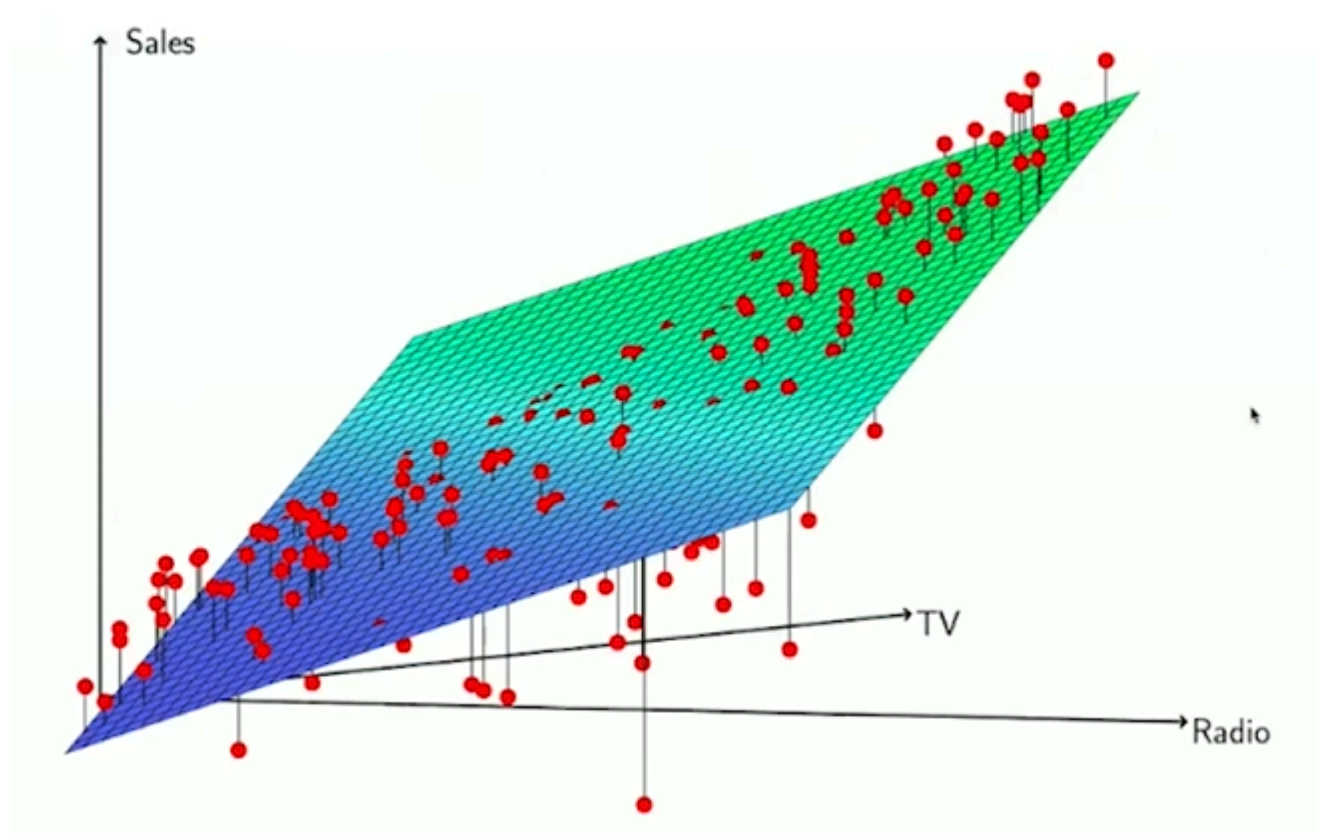
- 최소화 대상 RSS

$$\begin{aligned} RSS &= (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) \\ &= \mathbf{y}^\top \mathbf{y} - \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X} \hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} \end{aligned} \xrightarrow{\text{미분}} \frac{\partial RSS}{\partial \hat{\boldsymbol{\beta}}} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} = 0$$

- 정규방정식의 해

$$\hat{\boldsymbol{\beta}} = [\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p]^\top = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

3-4. 다중선형회귀 결과 : 시각화



3-5. 다중선형회귀 결과 : 광고 데이터

	Coefficient	Std. Error	t-statistic	p-value
Intercept	2.939	0.3119	9.42	< 0.0001
TV	0.046	0.0014	32.81	< 0.0001
radio	0.189	0.0086	21.89	< 0.0001
newspaper	-0.001	0.0059	-0.18	0.8599

Quantity	Value
Residual Standard Error	1.69
R^2	0.897
F-statistic	570

2. AI & 기계학습 방법론2 로지스틱 회귀(Logistic Regression)

1. 분류

1-1. 분류(Classification)란

- 범주형(질적) 변수는 순서가 없는 집합 c에서 값을 가짐
 - ex) 눈색 = {갈색, 검은색, 빨간색}
- 특징벡터 X와 질적 반응 Y 가 주어졌을 때, 입력 X를 받아 Y의 값을 예측하는 분류함수 $f(X)$ 를 학습하는 것이 목표 ($f(x)$ 는 집합 c에 속함)
- 실제로는 각 범주에 속할 확률 P 추정이 더 유용할 때가 많음
 - ex) 보험 사기 여부를 Yes/No로 분류하는 것보다, 사기일 확률 자체를 추정하는 것이 실무적으로 가치가 큼

1-2. 예시: 신용카드 연체(Default)

- Balance-Income 산점도에 연체 여부 색상으로 표시 (scatterplot)
- Balance, Income에 대해 연체 (Yes/No) 그룹별 분포 (boxplot)

1-3. 분류 문제에 선형 회귀를 써도 될까?

- 이진 분류 문제에서 Y {0,1} 로 부화하 하고 ,Y를 X에 대한 선형 회귀로 적합한 뒤, $\hat{Y} > 0.5$ 이상이면, Yes로 분류하는 단순 아이디어 가능
- 하지만 회귀 예측값을 확률로 해석하면 0 미만 1 초과가 나올 수 있어 확률로서 타당하지 않음
→ Logistic Regression이 더 적절

1-4. 다중범주에서 선형회귀는 불가능

- ex) 응급실 환자 진단 { 1 : 복통, 2 : 치통, 3 : 생리통, 3 : 아싸게보린 }
 - 이런 정수 코딩은 인위적 순서를 암시, $1 \leftrightarrow 2$, $2 \leftrightarrow 3$ 의 거리 동일설까지 암묵적으로 가정하게 됨
- 따라서 다중 범주 문제에서는 선형회귀보다 Logistic Regression 또는 다른 분류 모델이 적합

2. 로지스틱 회귀

2-1. 로지스틱(Logistic) 회귀의 모형식

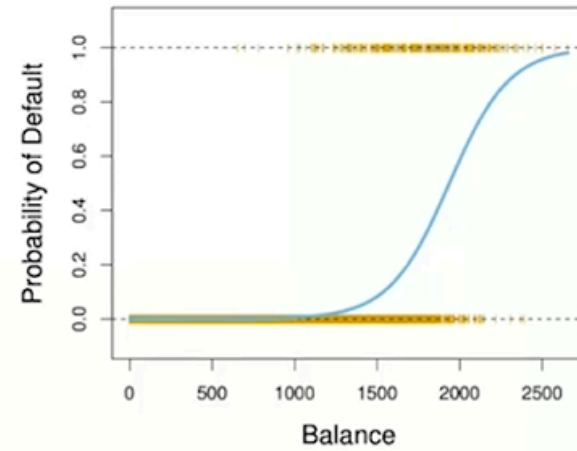
- $p(X) = P(Y = 1 | X)$ 로 두고, 로지스틱 회귀는

$$p(X; \beta) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}, \quad \beta = [\beta_0 \beta_1]^T,$$

으로 정의(오일러 수 $e \approx 2.71828$).

- 어떤 X, β 에 대해서도 $p(X; \beta) \in (0, 1)$ 가 보장됨.
- 이를 log를 이용하여 정리하면 (log-odds)

$$\log \frac{p(X; \beta)}{1 - p(X; \beta)} = \beta_0 + \beta_1 X$$



- X가 있을 때 Y의 값은 지수의 값이 크면 클 수록 1에 가까워짐
 - e = 오일러수
- 장점 : X가 아무리 크고 작아도 0~1 사이에 있음

2-2. MLE 활용 모수 추정

- 모수 추정은 MLE (Maximum Likelihood Estimation) 사용

$$\mathcal{L}(\beta) = \prod_{i: y_i=1} p(x_i; \beta) \prod_{i: y_i=0} (1 - p(x_i; \beta)), \quad p(x; \beta) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

- log-likelihood 로 바꾸면, 곱이 합으로 바뀌어 최적화가 쉬워짐

$$\log \mathcal{L}(\beta) = \sum_{i=1}^n [y_i \log p(x_i; \beta) + (1 - y_i) \log(1 - p(x_i; \beta))]$$

- $\log \mathcal{L}(\beta)$ 를 미분하여 0을 만족하는 해(도함수=0)를 수치적(반복) 최적화로 구함 (닫힌형 해는 없음).

$$\sum_i x_i \{y_i - p(x_i; \beta)\} = 0, \quad \sum_i \{y_i - p(x_i; \beta)\} = 0$$

2-3. 로지스틱 회귀 결과 : 신용카드 연체 데이터

- 신용카드 사용량(balance)과 연체 여부(default)로 로지스틱회귀 모형을 학습한 결과

	Coefficient	Std. error	z-statistic	p-value
Intercept	-10.6513	0.3612	-29.5	<0.0001
balance	0.0055	0.0002	24.9	<0.0001

- 추정 결과 $\hat{\beta}_1 = 0.0055$, 즉 **Balance가 1 단위 증가할 때 연체(Default)의 log-odds가 0.0055 증가**한다는 뜻.

$$\log \frac{p(X; \hat{\beta})}{1 - p(X; \hat{\beta})} = -10.6513 + 0.0055 \cdot X$$

$$X = 1000 \text{ 일 때}$$

$$\hat{p}(X) = \frac{e^{-10.6513+0.0055 \cdot 1000}}{1 + e^{-10.6513+0.0055 \cdot 1000}} = 0.006$$

$$X = 2000 \text{ 일 때}$$

$$\hat{p}(X) = \frac{e^{-10.6513+0.0055 \cdot 2000}}{1 + e^{-10.6513+0.0055 \cdot 2000}} = 0.586$$

- 여러 변수 $X_1, \dots, X_p$ 를 함께 쓰면:

$$\log \frac{p(X; \beta)}{1 - p(X; \beta)} = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

$$p(X; \beta) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}$$

	Coefficient	Std. error	z-statistic	p-value
Intercept	-10.8690	0.4923	-22.08	<0.0001
balance	0.0057	0.0002	24.74	<0.0001
income	0.0030	0.0082	0.37	0.7115
student [Yes]	-0.6468	0.2362	-2.74	0.0062

3. AI&기계학습 방법론 3 신경망모델 (Neural Network)

1. 선형 회귀

1-1. 1D 선형회귀

- 손실함수 : 최소제곱 손실함수

손실함수:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$

$$= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

1-2. 1D 선형회귀 학습

- 경사하강(Gradient descent)을 활용한 선형 회귀

2. Shallow 네트워크

2-1. Shallow 네트워크 vs 1D 선형회귀

- Shallow 네트워크 : 은닉층(hidden layer) 가 1개인 뉴럴 네트워크 (얇은 신경망)

1D 선형회귀

$$y = f[x, \phi] \\ = \phi_0 + \phi_1 x$$

Shallow 네트워크의 예

$$y = f[x, \phi] \\ = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

2-2. Shallow 네트워크 : 활성화 함수

활성화 함수(activation function)

- ReLU가 대표적
 - 음수는 0, 양수는 그대로 출력

$$y = f[x, \phi] \\ = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

활성화 함수 (activation function)

2-3. Shallow 네트워크 : 모수(parameter)

$$y = f[x, \phi] \\ = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

위 네트워크 모델은 10개의 모수를 갖고 있음

$$\phi = \{ \phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31} \}$$

학습을 한다면는 것은 파라미터를 구하는 것

- 파라미터가 정해지면, 특정 함수가 결정된다.
- 파라미터가 주어지면, 추론을 할 수 있다.
- 훈련 데이터가 주어지면, 손실 함수(least squares) 를 정의하고 손실을 최소화하도록 파라미터를 조정한다.

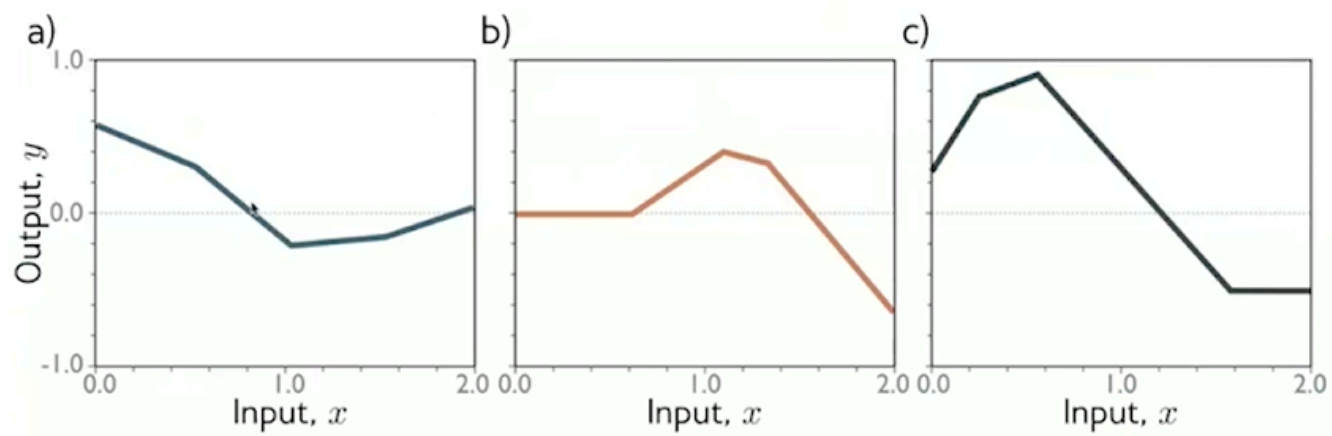
2-4. Shallow 네트워크 : piecewise linear 함수

Piecewise linear 함수 : 전체는 비선형이지만, 구간별로 선형인 함수

- 입력 구간을 나눠 조각별 선형 함수를 만들

$$y = f[x, \phi]$$

$$= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$



2-5. Shallow 네트워크 : Hidden Units

Hidden Units : 중간 단계의 연산

$$y = f[x, \phi]$$

$$= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

두 부분으로 나눠 생각:

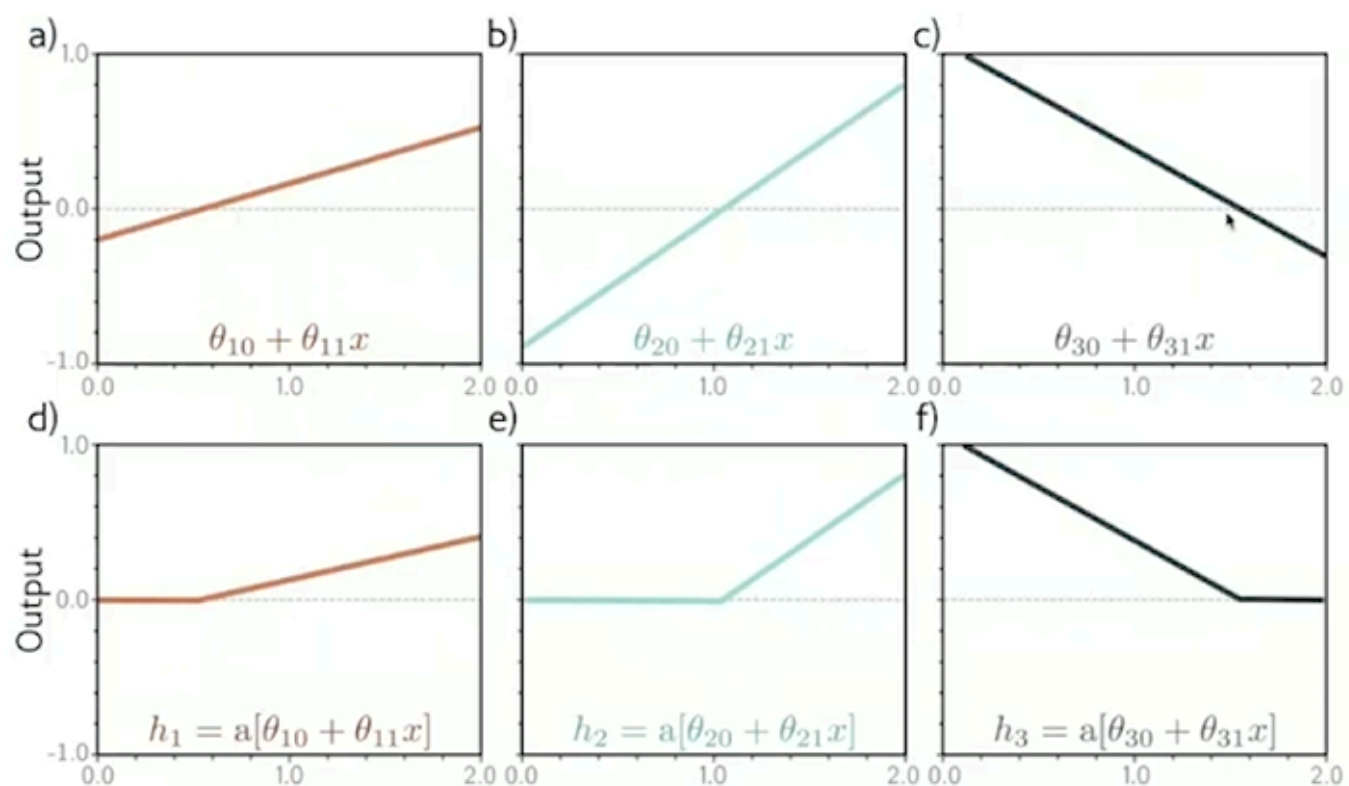
$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Hidden units

$$\begin{cases} h_1 = a[\theta_{10} + \theta_{11}x] \\ h_2 = a[\theta_{20} + \theta_{21}x] \\ h_3 = a[\theta_{30} + \theta_{31}x] \end{cases}$$

2-6. Shallow 네트워크 : 각 단계별 계산

- 각 hidden units 별 계산 → 활성화 함수 적용
- Hidden unit의 개수만큼 꺾임



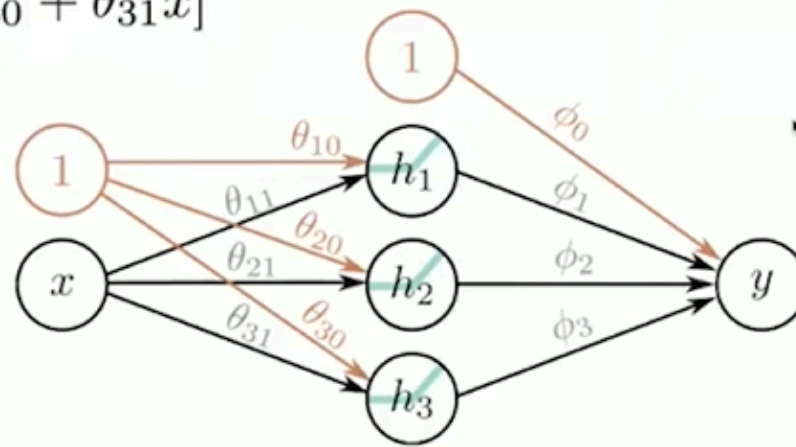
2-7. 네트워크 도식화

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



- 각 파라미터는 (출발 노드의 값) X (가중치) 를 도착 노드에 더한다.

3. Shallow 네트워크의 표현력

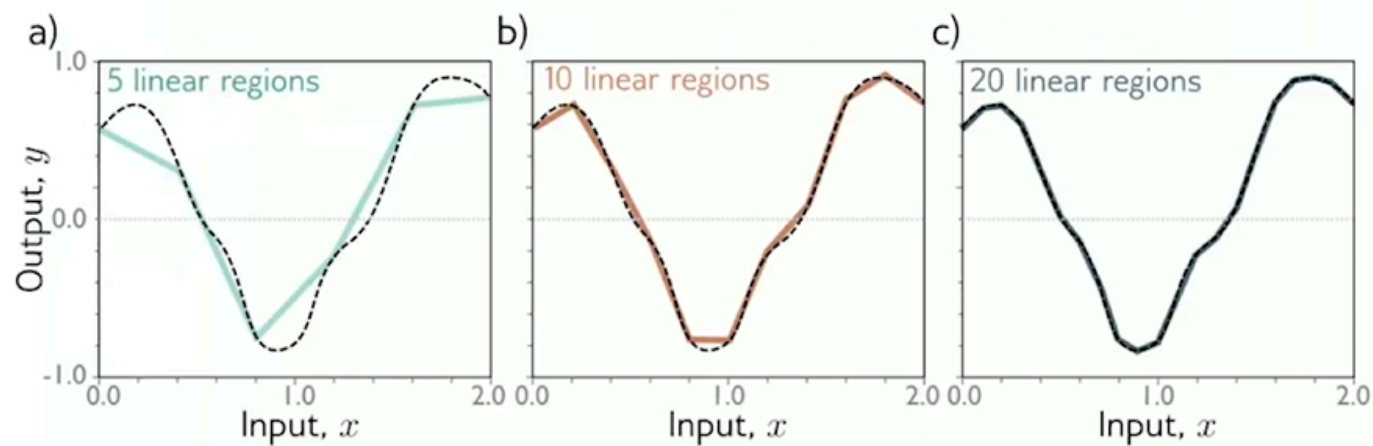
3-1. 더 많은 Hidden Unit 가능

Hidden layer 가 더 많아 질 수 있다. → 그래프에서 더 꺾임

- 3개 → D개

3-2. Hidden Unit을 많이 두면

- 임의의 1차원 함수를 원하는 정확도로 근사할 수 있다.



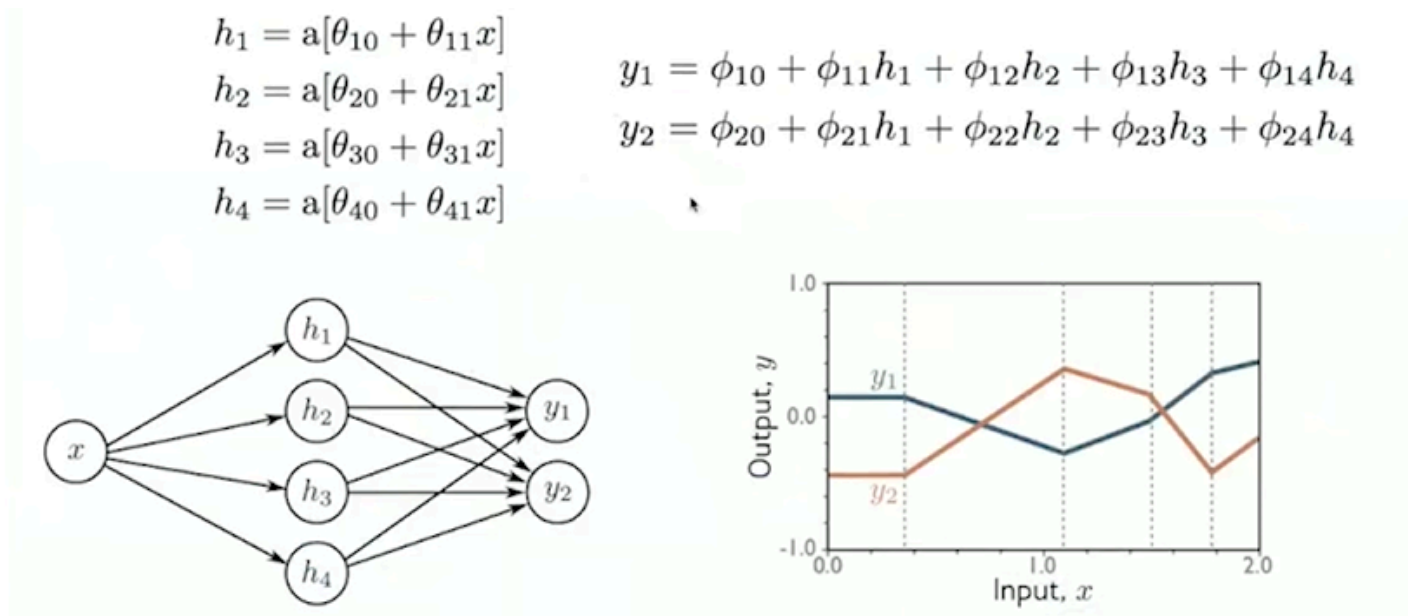
3-3. Universal approximation theorem(보편 근사정리)

- 은닉층이 1개인 Shallow NN 일지라도, Hidden unit을 충분히 많이 가지면, 콤팩트 부분집합(compact subset) 위에서 정의된 임의의 연속함수를 원하는 정밀도(e-precision)로 근사 할 수 있음이 증명 가능함

4. 다중 출력/입력

4-1. 2개 출력

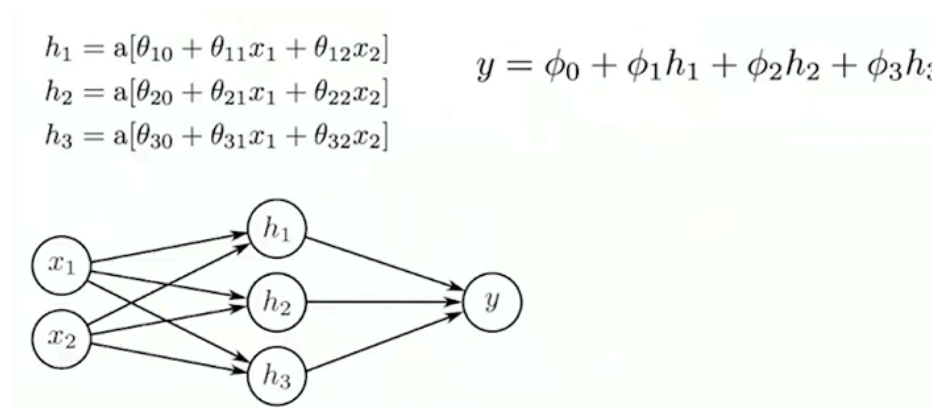
- 1 input , 4 hidden units, 2 outputs



- output이 여러 개여도, 꺾인 선의 위치는 동일 (Hidden unit에 결정)

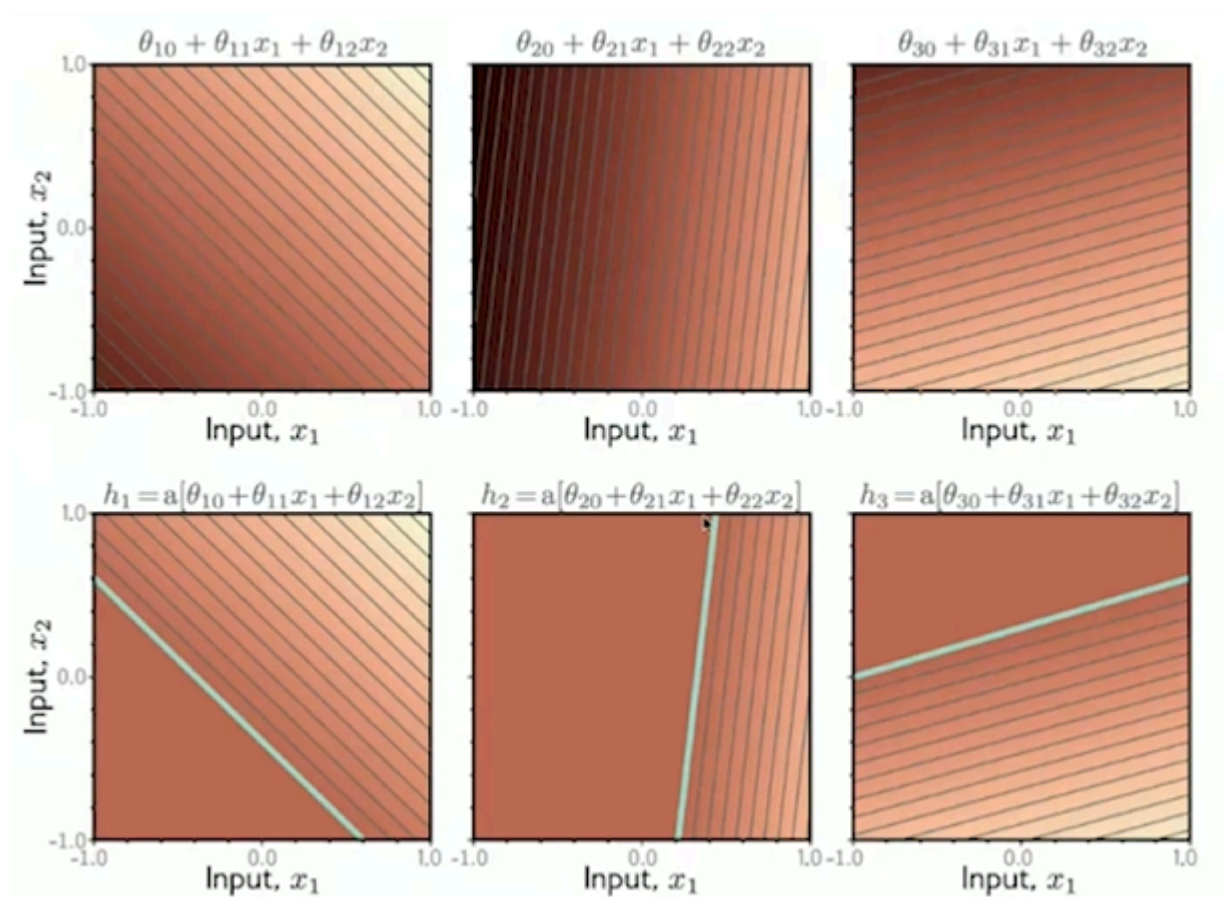
4-2. 2개 입력

- 2 input, 3 hidden units, 1 output

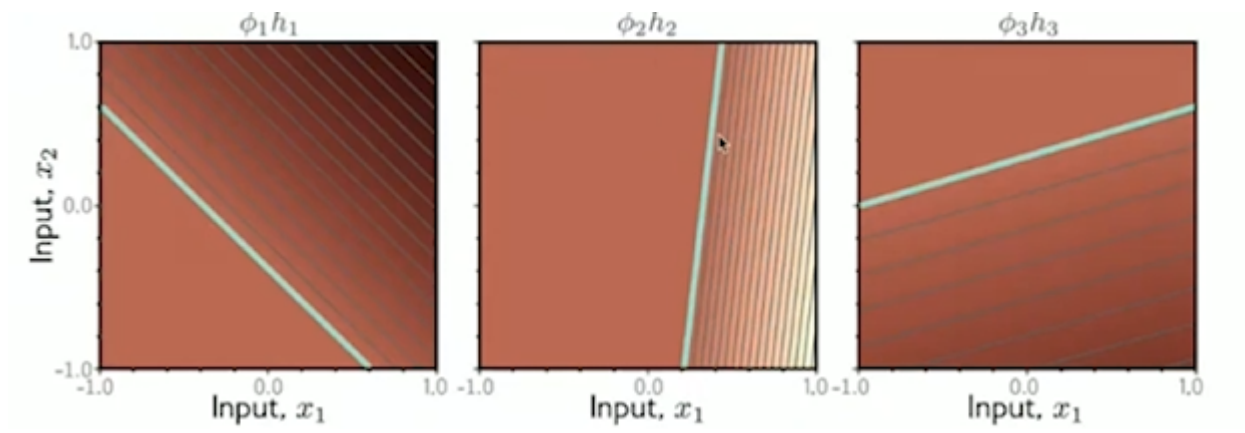


4-3. 2개 입력: 단계별 계산

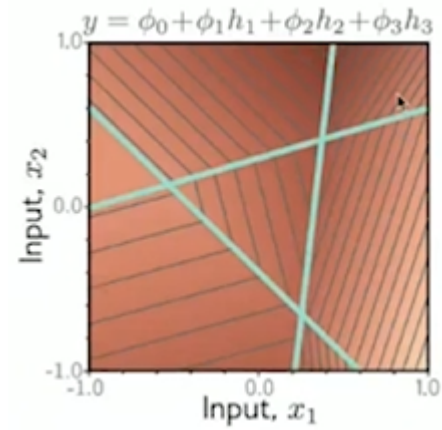
- input 2개
 - 2차식에 대한 선형 함수 → 평면
- 아래는 활성화 함수 대입



- 함수 대입

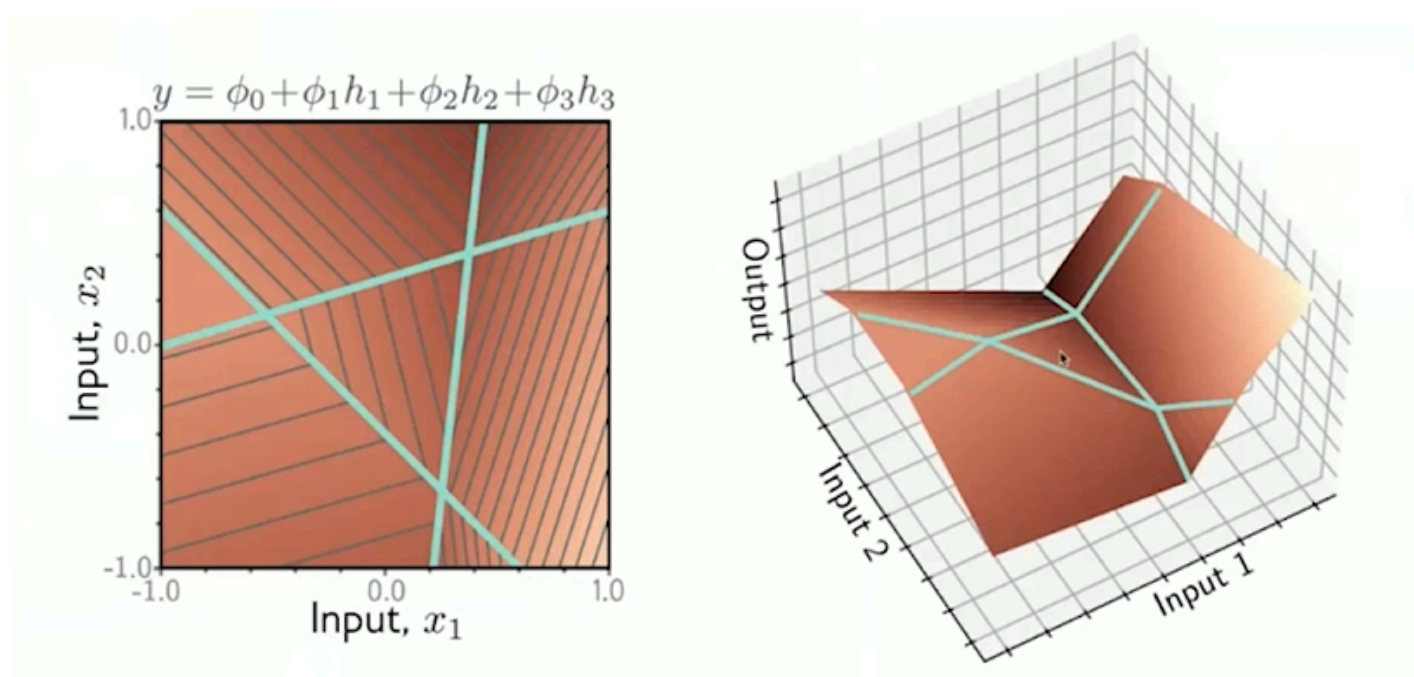


- 3개의 결과를 더한다.



4-4. 2개 입력 : From Input to Output

- 3차원으로 보면 접힌 것처럼 표현
 - [참고] ReLU 가 아니라 곡선 함수라면, 접힌 게 아니라 말린 것처럼 표현된다.

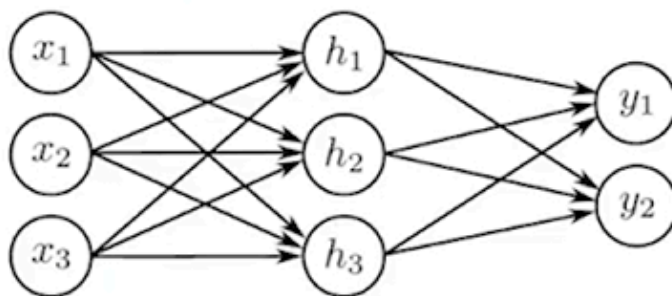


4-5. 임의의 개수의 입력, Hidden Unit, 출력

- Do Outputs, D hidden units, and Di inputs 일 때 일반화 가능

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right] \quad y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$

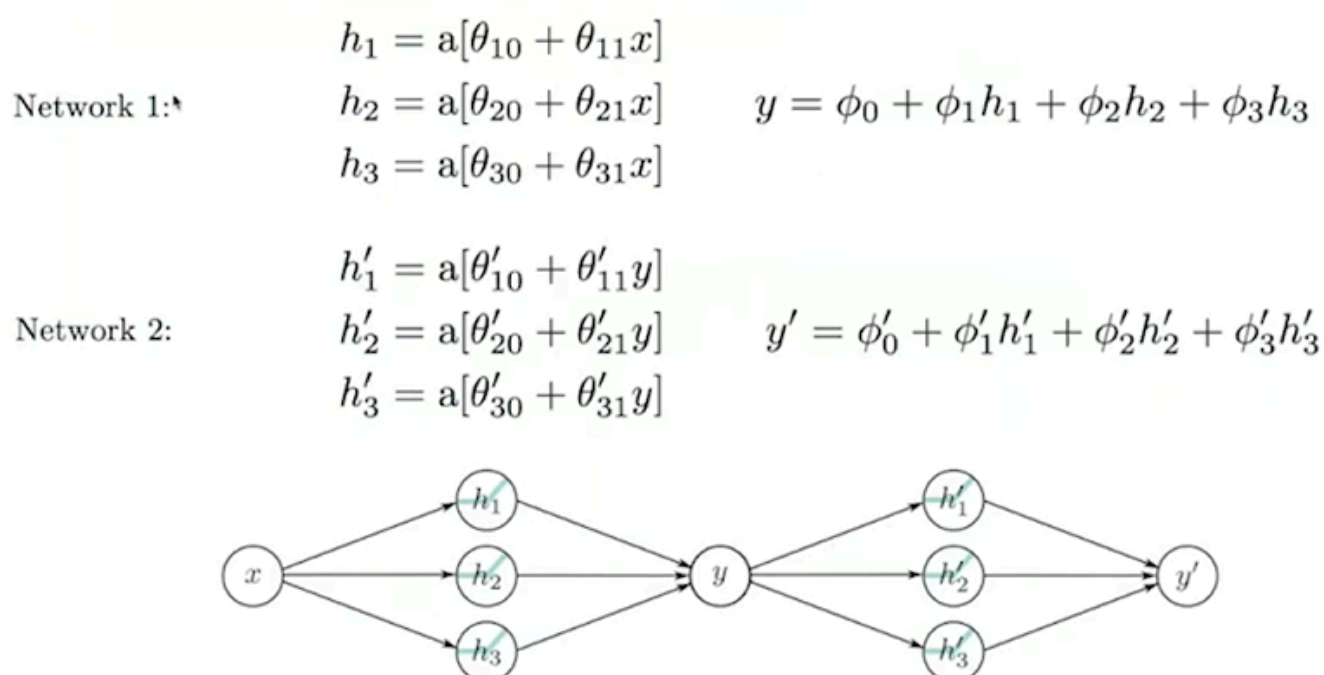
- 예, 3 inputs, 3 hidden units, 2 outputs



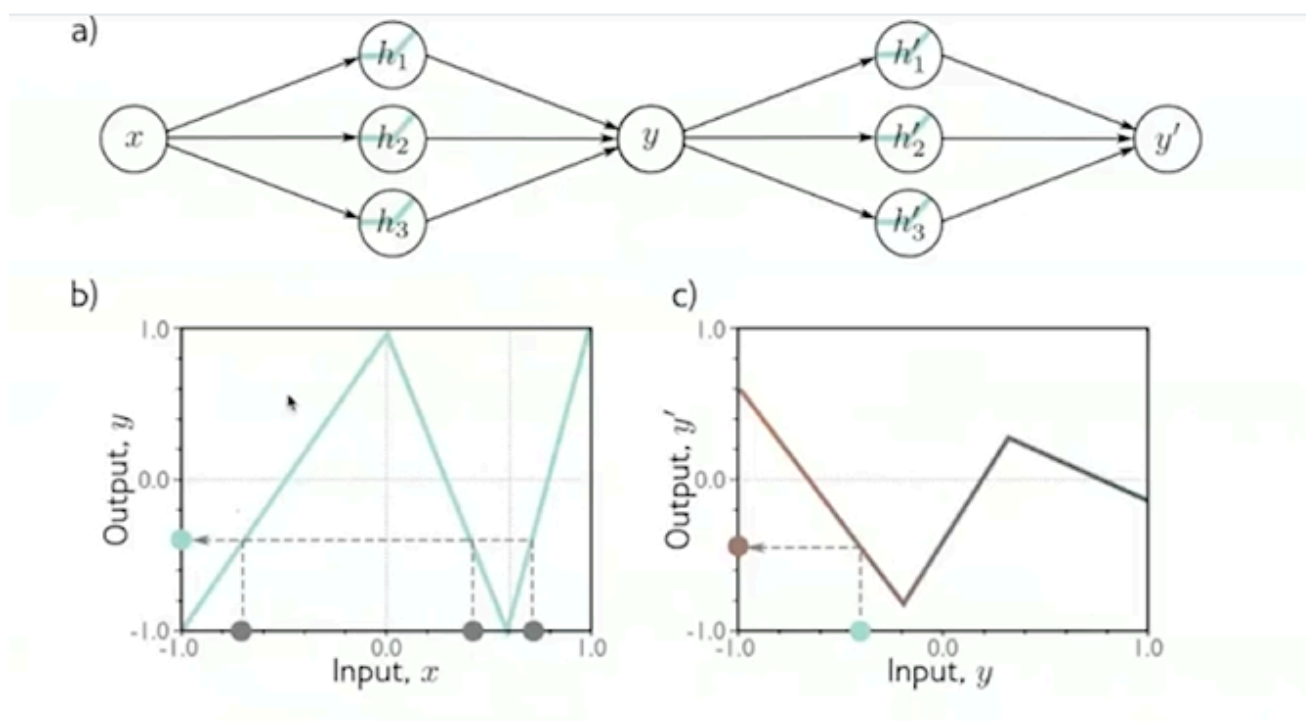
5. Deep 네트워크

Deep 네트워크 : 은닉층(hidden layer)가 2개 이상인 뉴럴네트워크

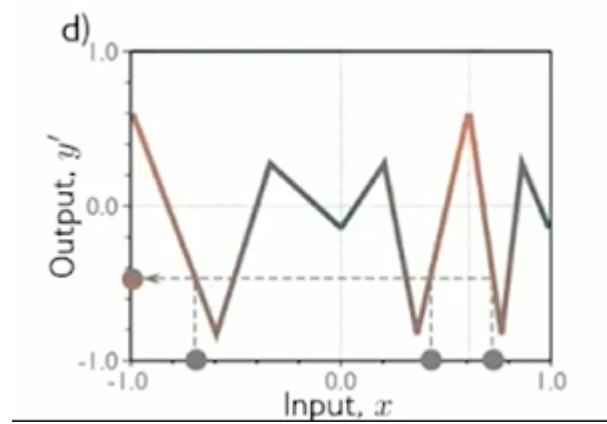
5-1. 2개의 Shallow 네트워크를 하나로 합성에서 시작



- 두 개의 shallow 네트워크의 hidden units 은 3개

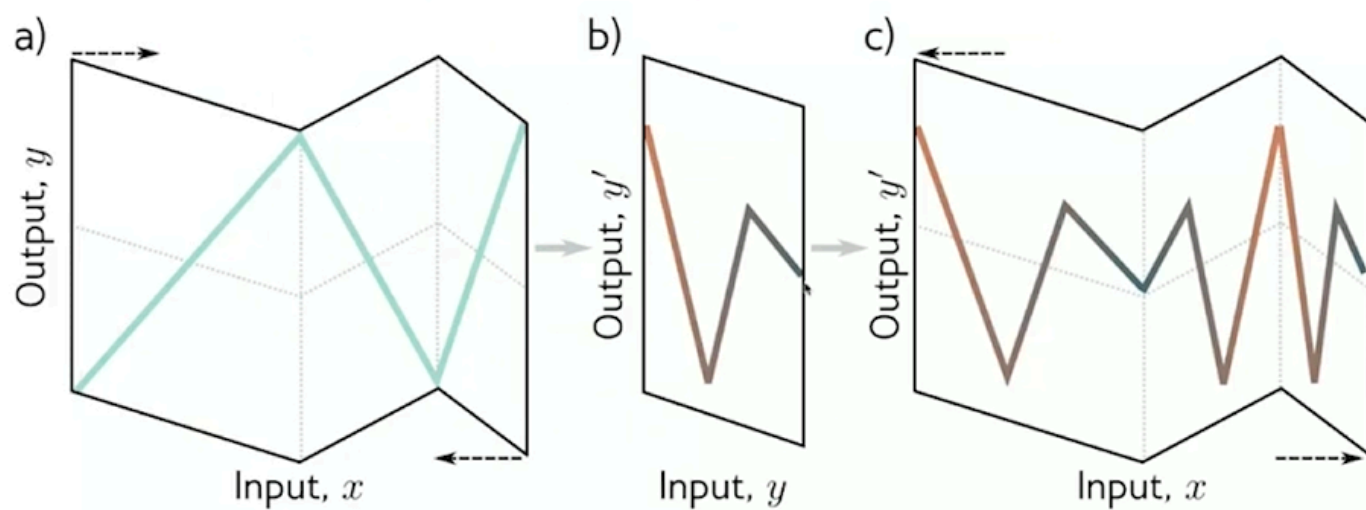


- b)의 그림 : 왼쪽 1번 Shallow NN
- c)의 그림 : 1번 Shallow NN 을 거쳐 나온 결과들이 2번 Shallow NN을 통과하는 과정



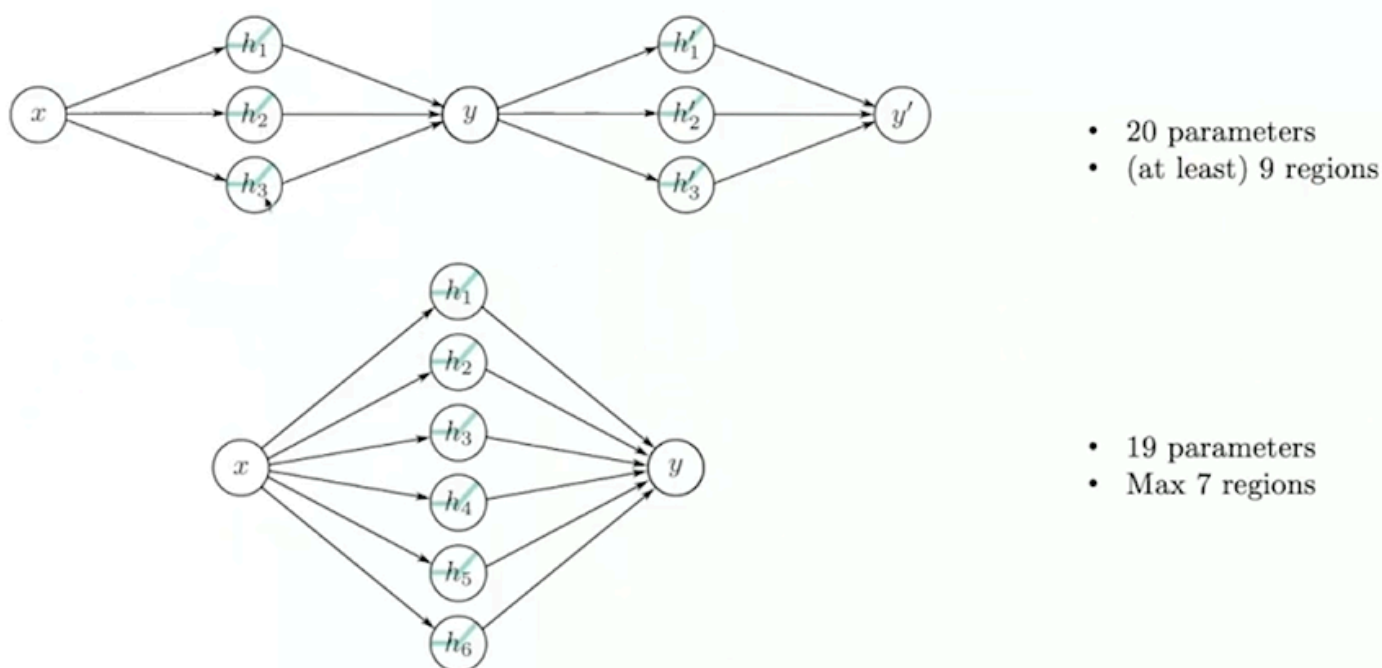
- b,c 의 결과
- 위 결과를 자세히 보면?!
 - 접기와 같다.

“접기 비유”



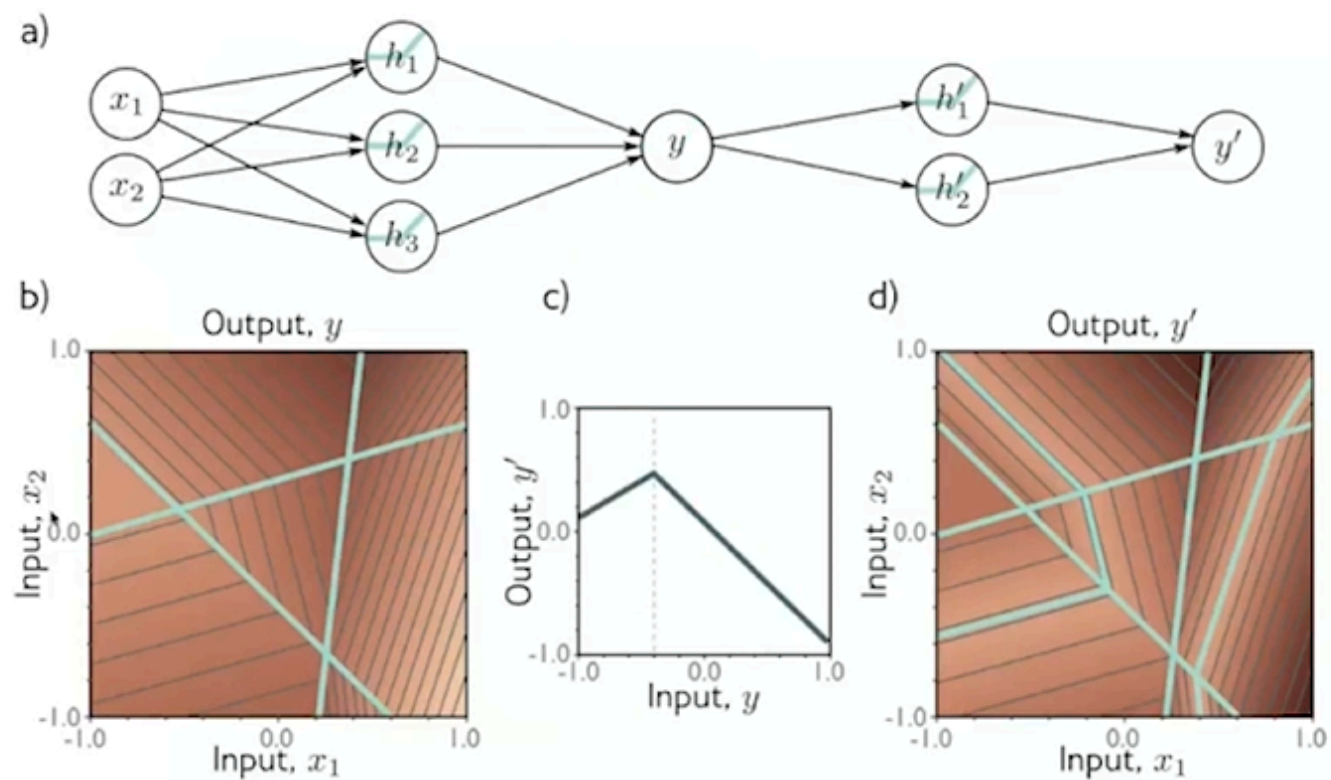
5-2. 6개의 Hidden Units의 Shallow NN과 비교

→ 3개의 hidden layer를 두개 겹친 게, 6개의 hidden layer를 1개 통과하는 것보다 더 많은 표현을 할 수 있다.



5-3. 2개의 네트워크를 하나로 합성

- input : 2개
- hidden : 3, 2개
- output : 1개



5-3. 2개의 네트워크를 하나로 합성

- 2개의 네트워크를 합쳐 하나의 더 큰 신경망으로 표현할 수 있다.
 - 표현력이 증가
- 다만, 수식이 자꾸 복잡해진다.

$$\begin{aligned}
 \text{Network 1:} \quad & h_1 = a[\theta_{10} + \theta_{11}x] \\
 & h_2 = a[\theta_{20} + \theta_{21}x] \\
 & h_3 = a[\theta_{30} + \theta_{31}x] \\
 & y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3
 \end{aligned}$$

$$\begin{aligned}
 \text{Network 2:} \quad & h'_1 = a[\theta'_{10} + \theta'_{11}y] \\
 & h'_2 = a[\theta'_{20} + \theta'_{21}y] \\
 & h'_3 = a[\theta'_{30} + \theta'_{31}y] \\
 & y' = \phi'_0 + \phi'_1 h'_1 + \phi'_2 h'_2 + \phi'_3 h'_3
 \end{aligned}$$

Hidden units of second network in terms of first:

$$\begin{aligned}
 h'_1 &= a[\theta'_{10} + \theta'_{11}y] = a[\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1 h_1 + \theta'_{11}\phi_2 h_2 + \theta'_{11}\phi_3 h_3] \\
 h'_2 &= a[\theta'_{20} + \theta'_{21}y] = a[\theta'_{20} + \theta'_{21}\phi_0 + \theta'_{21}\phi_1 h_1 + \theta'_{21}\phi_2 h_2 + \theta'_{21}\phi_3 h_3] \\
 h'_3 &= a[\theta'_{30} + \theta'_{31}y] = a[\theta'_{30} + \theta'_{31}\phi_0 + \theta'_{31}\phi_1 h_1 + \theta'_{31}\phi_2 h_2 + \theta'_{31}\phi_3 h_3]
 \end{aligned}$$

5-4. 새로운 변수 활용

- 두 개의 네트워크를 합성했을 때 수식이 복잡해지는 문제를 해결
- 새로운 변수 ψ 의 도입
 - 모든 은닉 유닛은 결국 첫번째 네트워크의 은닉 유닛들에 대한 새로운 선형 결합으로 표현
 - 네트워크를 합성해도 여전히 하나의 신경망 구조로 볼 수 있다.

$$\begin{aligned}
h'_1 &= a[\theta'_{10} + \theta'_{11}y] = a[\theta'_{10} + \theta'_{11}\phi_0 + \theta'_{11}\phi_1h_1 + \theta'_{11}\phi_2h_2 + \theta'_{11}\phi_3h_3] \\
h'_2 &= a[\theta'_{20} + \theta'_{21}y] = a[\theta'_{20} + \theta'_{21}\phi_0 + \theta'_{21}\phi_1h_1 + \theta'_{21}\phi_2h_2 + \theta'_{21}\phi_3h_3] \\
h'_3 &= a[\theta'_{30} + \theta'_{31}y] = a[\theta'_{30} + \theta'_{31}\phi_0 + \theta'_{31}\phi_1h_1 + \theta'_{31}\phi_2h_2 + \theta'_{31}\phi_3h_3]
\end{aligned}$$

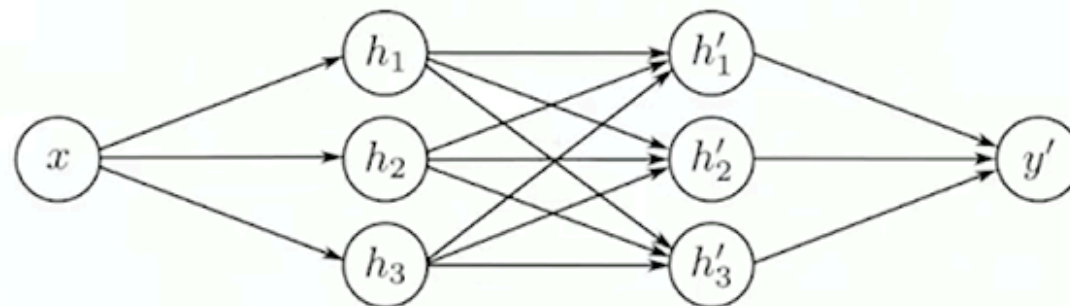
$$\begin{aligned}
h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\
h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\
h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3]
\end{aligned}$$

5-5. 2층 네트워크

- 두 개의 네트워크를 합쳐 표현한 결과이지만, 여전히 하나의 깊은 신경망으로 단순하게 표현 가능

$$\begin{aligned}
h_1 &= a[\theta_{10} + \theta_{11}x] & h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\
h_2 &= a[\theta_{20} + \theta_{21}x] & h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\
h_3 &= a[\theta_{30} + \theta_{31}x] & h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3]
\end{aligned}$$

$$y' = \phi'_0 + \phi'_1h'_1 + \phi'_2h'_2 + \phi'_3h'_3$$



5-6. 2층 네트워크를 하나의 식으로 표현

$$\begin{aligned}
h_1 &= a[\theta_{10} + \theta_{11}x] & h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\
h_2 &= a[\theta_{20} + \theta_{21}x] & h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\
h_3 &= a[\theta_{30} + \theta_{31}x] & h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3]
\end{aligned}$$

$$y' = \phi'_0 + \phi'_1h'_1 + \phi'_2h'_2 + \phi'_3h'_3$$

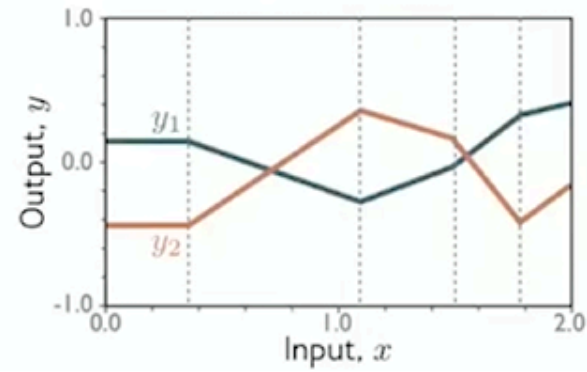
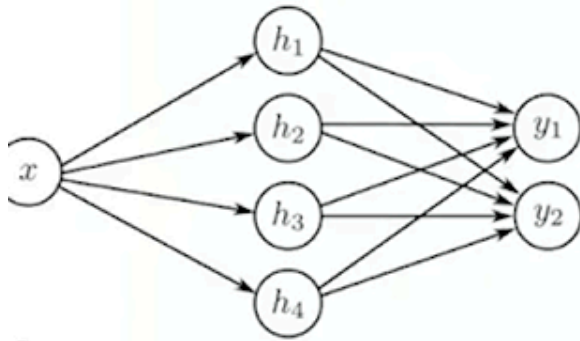
$$\begin{aligned}
y' &= \phi'_0 + \phi'_1a[\psi_{10} + \psi_{11}a[\theta_{10} + \theta_{11}x] + \psi_{12}a[\theta_{20} + \theta_{21}x] + \psi_{13}a[\theta_{30} + \theta_{31}x]] \\
&\quad + \phi'_2a[\psi_{20} + \psi_{21}a[\theta_{10} + \theta_{11}x] + \psi_{22}a[\theta_{20} + \theta_{21}x] + \psi_{23}a[\theta_{30} + \theta_{31}x]] \\
&\quad + \phi'_3a[\psi_{30} + \psi_{31}a[\theta_{10} + \theta_{11}x] + \psi_{32}a[\theta_{20} + \theta_{21}x] + \psi_{33}a[\theta_{30} + \theta_{31}x]]
\end{aligned}$$

5-7. 2개 output의 shallow 네트워크

- 1 input, 4 hidden units, 2 outputs

$$\begin{aligned}h_1 &= a[\theta_{10} + \theta_{11}x] \\h_2 &= a[\theta_{20} + \theta_{21}x] \\h_3 &= a[\theta_{30} + \theta_{31}x] \\h_4 &= a[\theta_{40} + \theta_{41}x]\end{aligned}$$

$$\begin{aligned}y_1 &= \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4 \\y_2 &= \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4\end{aligned}$$



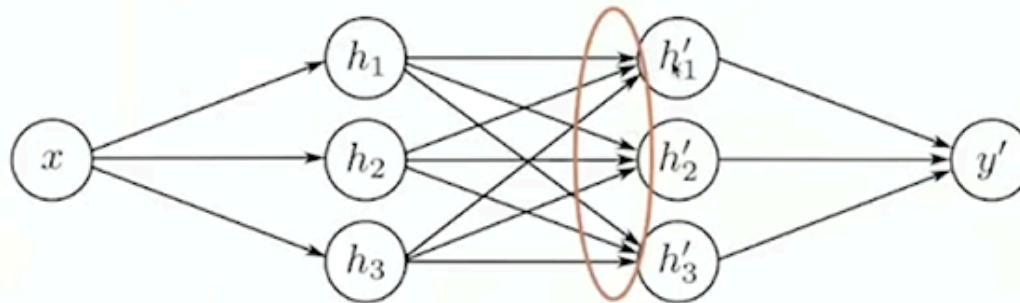
5-8. 네트워크를 합성함수로!

- 2개의 네트워크를 합치면 사실상 단일 네트워크의 선형 변환 + 비선형 변환으로 표현 가능

$$\begin{aligned}h_1 &= a[\theta_{10} + \theta_{11}x] \\h_2 &= a[\theta_{20} + \theta_{21}x] \\h_3 &= a[\theta_{30} + \theta_{31}x]\end{aligned}$$

$$\begin{aligned}h'_1 &= a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\h'_2 &= a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\h'_3 &= a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3]\end{aligned}$$

Consider the pre-activations at the second hidden units
At this point, it's a one-layer network with three outputs



경사하강법

알고리즘

- 손실함수 L 을 파라미터 ϕ 에 대해 미분 (기울기 계산)

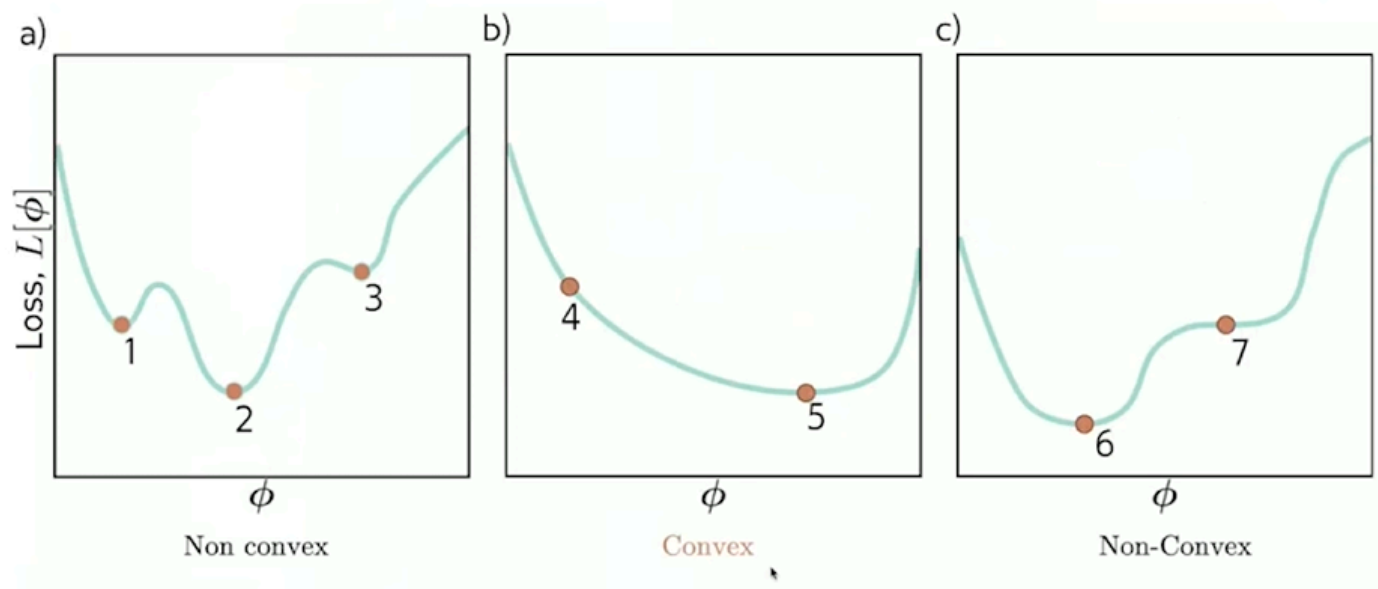
$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}.$$

- 다음 규칙에 따라 파라미터 업데이트

$$\phi \leftarrow \phi - \alpha \frac{\partial L}{\partial \phi},$$

α 는 양수 상수로, 변화의 크기(학습률, learning rate)를 결정한다.

Convex problems



- **Convex 문제**
 - 전역 최소점이 유일 → 최적화가 쉽다.
 - 선형 회귀 같은 문제는 Convex 문제.
- **Non-convex 문제**
 - 여러 지역 최소점이 존재 → 전역 최적점을 보장하기 어려움.
 - 신경망 학습은 대부분 Non-convex 문제.

아이디어 : 노이즈 추가

- 전체 데이터로 기울기 계산하지 않고, 일부만 사용해 계산
→ 계산이 빨라지고, 기울기에 노이즈가 생겨, local minima(지역 최소값)에서 벗어나는 데 도움을 줌

SGD (Stochastic Gradient Descent)

- 한번에 데이터 하나만 사용해 기울기 계산
- 장점 : 계산이 빠르고, 지역 최소에 빠질 확률이 적음
- 단점 : 기울기 추정이 매우 불안정 → 진동 심함

Minibatch-SGD

- 데이터 전체 대신 일부(미니배치) 샘플을 사용해 기울기 계산
- 장점 : 계산 효율적(병렬 처리 가능) , SGD 보다 안정적, 전체 배치보다 빠름

Epoch

- 데이터 전체를 한번 학습하는 과정
- ex) 데이터 100개 , 배치 크기 100이면, epoch → 10번의 업데이트로 구성